



11. Dominando funções Python

☑ Dúvidas?	☑
☑ Exercícios a entregar?	☑
📁 Módulo	2 - Dominando funções e boas práticas em Python
☰ Tipo de aula	Teste Vídeo
≡ Links e mídias	Pasta "Aula 11"
🕒 Última atualização	@18 de novembro de 2025 04:41
⚙ Status	Done
📅 Data início	@17/11/2025
📅 Data conclusão	@18/11/2025 2:30
⌚ Esforço	Médio

Anotações

Vídeo 1 - Funções Python (parte 1)

O que são funções?

Retorno de valores

Parâmetros nomeados

Args e kwargs

Vídeo 2 - Funções Python (parte 2)

[Parâmetros especiais](#)

[Objetos de primeira classe](#)

[Escopos local e global](#)

[Exercícios](#)

[Arquivos](#)

Anotações

Vídeo 1 - Funções Python (parte 1)

O que são funções?

Uma função é um **bloco de código** identificado por um **nome** (o **identificador**), e pode receber uma lista de **parâmetros**, que podem ou não ter **valores padrão**. Usar funções torna o código mais legível e possibilita o reaproveitamento do código. Programar usando funções é o mesmo que dizer que estamos programando de maneira estruturada.

Os parâmetros são o que enviamos como **entrada** para a função.

A função também pode **retornar valores** e, no caso do Python, pode retornar até mais de um valor. Esses retornos são as **saídas** da função.

Exemplo:

```
def exibir_mensagem(): # sem parâmetros
    print("Olá, mundo!")

def exibir_mensagem_2(nome): # com parâmetro "nome"
    print(f"Seja bem-vindo, {nome}!")

def exibir_mensagem_3(nome="Anônimo"): # com parâmetro "nome" com
    um valor padrão
    print(f"Seja bem-vindo, {nome}!")

exibir_mensagem()
exibir_mensagem_2(nome="Lizandra") # OU exibir_mensagem_2("Lizandra"),
# se não fornecermos um nome aqui, ocorre um TypeError
exibir_mensagem_3()
```

```
exibir_mensagem_3(nome="Lizandra") # se alterarmos o padrão, é exibida a alteração
```

`def` é uma palavra reservada no Python para criar funções, ela mostra ao interpretador Python que o nome a seguir não é uma variável, mas sim o nome de uma função.



Declarar uma função com `def` somente não faz com que ela seja executada. Para que ela funcione, é preciso chamá-la posteriormente no código pelo seu identificador.

Retorno de valores

Para que a função retorne um valor, é preciso usar a palavra reservada `return`. Toda função Python retorna `None` por padrão. Diferente de outras linguagens de programação, em Python uma função pode retornar mais de um valor.

Exemplo:

```
def calcular_total(numeros):  
    return sum(numeros)  
  
def retorna_antecessor_e_sucessor(numero):  
    antecessor = numero - 1  
    sucessor = numero + 1  
  
    return antecessor, sucessor  
  
print(calcular_total([10, 20, 34])) # 64  
print(retorna_antecessor_e_sucessor(10)) # (9, 11), retorna uma tupla porque ela é imutável  
  
print(calcular_total([6, 6, 1979])) # 1991  
print(retorna_antecessor_e_sucessor(8)) # (7, 9)
```

```
def func_3():  
    print("Olá, mundo!")  
  
print(func_3()) # retorna `None`
```

Parâmetros nomeados

Também podemos chamar funções usando **parâmetros nomeados**, no formato `chave=valor`. Foi o que fizemos no exemplo acima com a definição de `"Anônimo"` como `"nome"`.

Exemplo:

```
def salvar_carro(marca, modelo, ano, placa):  
    # código para salvar um carro no banco de dados...  
    print(f"Carro inserido com sucesso! {marca}/{modelo}/{ano}/{placa}")  
  
salvar_carro("Fiat", "Palio", 1999, "ABC-1234")  
salvar_carro(marca="Fiat", modelo="Palio", ano=1999, placa="ABC-1234")  
salvar_carro(**{"marca": "Fiat", "modelo": "Palio", "ano": 1999, "placa": "ABC-1234"})
```

As três execuções da função `salvar_carro` exibirão o mesmo resultado. A diferença no primeiro caso é que, se o usuário trocar o lugar das informações, a exibição sairá errada, por exemplo:

```
salvar_carro("Palio", "Fiat", 1999, "ABC-1234")  
# será exibido como "Carro inserido com sucesso! Palio/Fiat/1999/ABC-1234"
```

Neste caso, também, se alguém alterar a ordem dos parâmetros na declaração da função, novamente a exibição do resultado sairá incorreta.

O segundo caso também pode apresentar erros se alguém alterar o nome dos parâmetros na declaração da função, por exemplo:

```
def salvar_carro(marca, mod, ano, placa):  
    # código para salvar um carro no banco de dados...  
    print(f"Carro inserido com sucesso! {marca}/{mod}/{ano}/{placa}")
```

```
salvar_carro(marca="Fiat", modelo="Palio", ano=1999, placa="ABC-1234")  
# isso gera um erro, dizendo que "modelo" não existe na função
```

Então, por consequência, a melhor opção é a terceira, que passa um dicionário para a função. Observe que **os asteriscos são obrigatórios** para que o interpretador entenda que isto é um dicionário.

Args e kwargs

Podemos combinar parâmetros obrigatórios com **args** e **kwargs**. Ao defini-los, o método recebe os valores como **tupla** e **dicionário**, respectivamente.

***args** = *arguments*, renderiza como uma tupla. Ao definir uma função com ***args**, ela pode receber quantos argumentos forem passados para ela na chamada. Todos esses argumentos são agrupados em uma tupla com o nome **args**.

****kwargs** = *keyword arguments*, renderiza como um dicionário. Coleta todos os argumentos passados para a função que não correspondem aos parâmetros nomeados definidos.



Observe que as palavras **args** e **kwargs** não são obrigatórias e podem ser substituídas por um identificador, como **valores** ou **numeros**. Porém, os asteriscos **são obrigatórios**.

args é usado para argumentos posicionais (sem nome) e ***kwargs** é usado para argumentos nomeados (com palavra-chave).

A ordem correta na definição da função é: primeiro argumentos regulares, depois **args** e, por último, ***kwargs**.

Exemplo:

```
def exibir_poema(data_extenso, *args, **kwargs):  
    texto = "\n".join(args) # isso concatena todos os args com uma nova linha,  
    gerando uma nova string  
    meta_dados = "\n".join([f"{chave.title()}: {valor}" for chave, valor in kwargs.items()])  
    # itera em uma lista de tuplas (kwargs.items()), que gera "chave" e "v
```

```

    alor", que são aplicados em uma nova
        # string, concatenados com uma nova linha
    mensagem = f"{data_extenso}\n\n{texto}\n\n{meta_dados}" # concatena
a data, o texto definido acima e
        # meta_dados definido acima, gerando uma nova string
    print(mensagem)

exibir_poema(
    "Segunda-feira, 17 de novembro de 2025", # parâmetros posicionais
    "Zen of Python",
    "Beautiful is better than ugly.",
    "Explicit is better than implicit.",
    "Simple is better than complex.",
    "Complex is better than complicated.",
    "Flat is better than nested.",
    "Sparse is better than dense.",
    "Readability counts.",
    "Special cases aren't special enough to break the rules.",
    "Although practicality beats purity.",
    "Errors should never pass silently.",
    "Unless explicitly silenced.",
    "In the face of ambiguity, refuse the temptation to guess.",
    "There should be one-- and preferably only one --obvious way to do it.",
    "Although that way may not be obvious at first unless you're Dutch.",
    "Now is better than never.",
    "Although never is often better than *right* now.",
    "If the implementation is hard to explain, it's a bad idea.",
    "If the implementation is easy to explain, it may be a good idea.",
    "Namespaces are one honking great idea -- let's do more of those!",
    autor="Tim Peters", # parâmetros nomeados
    ano=1999,
)

```

Observe que, neste caso, há vários parâmetros separados por vírgulas. O primeiro parâmetro sempre será considerado como `data_extenso`.

Como os valores seguintes, **que não são nomeados**, estão separados por vírgulas, são considerados como parte da tupla, que entra como `*args`.

E os valores **nomeados**, por conterem **chave** e **valor**, são considerados como parte do dicionário, que entra como `**kwargs`.

Vídeo 2 - Funções Python (parte 2)

Parâmetros especiais

Por padrão, podemos passar parâmetros para funções por **posição** ou pelo **nome**. Para melhor legibilidade e desempenho, faz sentido restringir a maneira de passar esses parâmetros, assim um desenvolvedor só precisa olhar para a definição da função para determinar se os itens devem ser passados **por posição, por posição e nome** ou **só por nome**.

No exemplo acima, temos parâmetros **nomeados** e **posicionais**. E também podemos **forçar** parâmetros nomeados e posicionais usando a seguinte sintaxe:

```
def function_name(pos1, pos2, /, pos_ou_nome, *, nome1, nome2):
```

-----		-----		-----	
Apenas posição					
		Posição ou nome			
				Apenas nomeados	

Exemplos:

Exemplo 1 - somente posição:

```
def criar_carro(modelo, ano, placa, /, marca, motor, combustivel):  
    print(modelo, ano, placa, marca, motor, combustivel)
```

```
criar_carro("Palio", 1999, "ABC-1234", marca="Fiat", motor="1.0", combustivel="Gasolina")
```

```
# válido, Palio 1999 ABC-1234 Fiat 1.0 Gasolina
```

```
criar_carro(modelo="Palio", ano=1999, placa="ABC-1234", marca="Fiat", motor="1.0", combustivel="Gasolina")
```

```
# inválido, exibe "TypeError: criar_carro() got some positional-only arguments passed as keyword arguments:
```

```
# 'modelo, ano, placa'"
```

```
criar_carro("Palio", 1999, "ABC-1234", "Fiat", "1.0", "Gasolina")  
# válido, Palio 1999 ABC-1234 Fiat 1.0 Gasolina
```

Exemplo 2 - posicionais e nomeados:

```
def criar_carro(modelo, ano, placa, /, *, marca, motor, combustivel):  
    print(modelo, ano, placa, marca, motor, combustivel)  
  
criar_carro("Palio", 1999, "ABC-1234", marca="Fiat", motor="1.0", combusti  
vel="Gasolina")  
    # válido, Palio 1999 ABC-1234 Fiat 1.0 Gasolina  
criar_carro(modelo="Palio", ano=1999, placa="ABC-1234", marca="Fiat", m  
otor="1.0", combustivel="Gasolina")  
    # inválido, gera um TypeError: criar_carro() got some positional-only arg  
uments passed as keyword arguments: 'modelo, ano, placa'
```

⚠ Sem esquecer que podemos fazer também o seguinte neste caso:

```
def criar_carro(modelo, ano, placa, /, marca, *, motor, combustivel):  
    print(modelo, ano, placa, marca, motor, combustivel)  
  
# E os dois casos abaixo serão válidos:  
criar_carro("Palio", 1999, "ABC-1234", marca="Fiat", motor="1.0", combusti  
vel="Gasolina")  
criar_carro("Palio", 1999, "ABC-1234", "Fiat", motor="1.0", combustivel="Ga  
solina")
```

Exemplo 3 - somente nomeados:

```
def criar_carro(*, modelo, ano, placa, marca, motor, combustivel):  
    print(modelo, ano, placa, marca, motor, combustivel)  
  
criar_carro(modelo="Palio", ano=1999, placa="ABC-1234", marca="Fiat", m  
otor="1.0", combustivel="Gasolina")  
    # válido, Palio 1999 ABC-1234 Fiat 1.0 Gasolina
```

Objetos de primeira classe

Em Python, TUDO é objeto; portanto **funções também são objetos**, e são consideradas objetos de **primeira classe**. Com isso, podemos **atribuir funções a variáveis**, **passá-las como parâmetros para funções**, **usá-las como valores em estruturas de dados** (listas, tuplas, dicionários, etc.) e **usá-las como valor de retorno para uma função** (conhecido como **closures**).

Exemplo:

```
# criação de uma função "somar"
def somar(a, b):
    return a + b

# criação de uma função "exibir_resultado"
def exibir_resultado(a, b, funcao): # "funcao, aqui, poderia ter qq nome
    resultado = funcao(a, b)
    print(f"O resultado de {a} + {b} = {resultado}")

# Note que a função "somar" não é executada, isto é não é chamada com
parênteses, só chamamos seu identificador
exibir_resultado(10, 10, somar) # O resultado da operação 10 + 10 = 20
```

💡 Fiz mais operações no VS Code.

? Fiquei com uma dúvida aqui: brincando no VS Code, criei as funções `subtrair`, `multiplicar` e `dividir` como no exemplo de `somar`. Tentei usar o método `round()` ao executar a função `exibir_resultado` e recebi um `TypeError: "type NoneType doesn't define round method"`. Por quê?

Também podemos atribuir a função a uma variável e usar essa variável no código posteriormente. Usando a mesma função `somar`:

```
operacao = somar

print(operacao(1, 23)) # 24
```

Escopos local e global

O Python trabalha com os conceitos de **escopo local** e **escopo global**. Dentro do bloco da função temos o escopo local, fora dele é o escopo global. O conceito é similar ao do JavaScript.

Alterações feitas no escopo local em objetos **imutáveis** serão perdidas quando o método terminar de ser executado. Porém, objetos **mutáveis** manterão suas alterações.

Para que o interpretador considere uma variável manipulada no escopo local como do escopo global, é preciso usar a palavra reservada `global`; entretanto **esta prática NÃO é recomendada**.

Exemplo:

```
salario = 2000 # variável com escopo global

def salario_bonus(bonus):
    global salario
    salario += bonus
    return salario

salario_bonus(500) # 2500
```

? Mais uma dúvida aqui: Se usar a palavra reservada `global` não é a prática recomendada, qual seria a prática recomendada neste caso? Ele menciona no vídeo usar a variável global como parâmetro na função, mas não deu exemplos. Alguém pode me dar um exemplo? E seria esta a prática recomendada?

Exercícios

100% na primeira tentativa ⇒ 7/7 corretas

Arquivos

Pasta "Aula 11"